

StarNet for CIFAR-10 Image Classification

CISC3024 Pattern Recognition | AI Assignment 1 | September 2026

This project adapts the CVPR 2024 StarNet CNN for small-image classification. Under a 20-epoch training budget, StarNet achieved 86.14% test accuracy, compared with 87.22% for a conventional CNN and 85.60% for the additive ablation. These results were obtained from local experiments on CIFAR-10 and do not reproduce the paper's ImageNet experiments.

1 How I asked AI to find the algorithm

Research prompts and approval sequence

I first asked Codex to research the assignment before implementing anything. These are verbatim excerpts from my initial message:

Read `requirement.txt` completely. Do not modify any files or write any code yet. First inspect the workspace and available computing resources, then research several recent Deep CNN / Deep Autoencoder algorithms that satisfy the assignment.

Do not start implementation until I approve the algorithm and project plan.

I requested a comparison covering original papers, publication years, suitability for the assignment, applications, datasets, implementation difficulty and computing requirements, followed by a justified recommendation. When the inspection was interrupted, I followed up with:

keep searching

After receiving the comparison, I explicitly approved the proposal:

I approve the project plan:

Algorithm: StarNet (Rewrite the Stars, CVPR 2024) Application: CIFAR-10 image classification

The same approval message requested the complete assignment, including experiments, a data-driven website and the report. It also stated:

Please do not fabricate any results. All reported numbers, tables, and figures must come from actual executed experiments.

The sequence was therefore research, approval and implementation. The implementation-stage notes separately record `read requirement_2.txt and execute it`, which refers to a later stage of the project. The earlier selection dialogue corrects the previous report's statement that no search conversation was available.

Research process

Codex read all thirteen sections of `requirement.txt` through TextEdit and inspected the workspace through Finder. The visible folder then contained only `requirement.txt`. System Settings identified an

Apple M5 MacBook Pro, 16 GB unified memory, macOS 26.6.2 and approximately 851 GB free storage. ML package availability and actual MPS execution were not established during this initial inspection; the later measured environment appears in section 4.

The Google search query was **FasterNet Run Don't Walk CVPR 2023 StarNet 2024 convolutional network**. Codex then visited the FasterNet arXiv record, the CVF publication pages for StarNet, ConvNeXt V2 and DRAEM, the official StarNet repository, and the official CIFAR and MVTec AD dataset pages. These were direct source visits rather than additional search-engine queries. The implementation-stage notes also document research on ConvNeXt and inspection of the official StarNet block. Model category, spatial downsampling and local feasibility were considerations in this process; they are not presented as verbatim student prompts.

Candidate and primary paper	Year and eligibility	Application and datasets	Difficulty and local feasibility
StarNet, <i>Rewrite the Stars</i> [1]	CVPR 2024; deep CNN	Object classification; CIFAR-10/CIFAR-100	Moderate; compact convolutional blocks with multiplicative branches and a direct operator ablation.
FasterNet, <i>Run, Don't Walk</i> [3]	CVPR 2023; deep CNN	Efficient recognition; CIFAR-10/CIFAR-100	Moderate; partial convolution reduces redundant work, but channel slicing and hardware-dependent latency need care.
ConvNeXt V2, <i>Co-Designing and Scaling ConvNets With Masked Autoencoders</i> [5]	CVPR 2023; deep CNN with convolutional masked-autoencoder pretraining	Classification and representation learning; reduced-scale CIFAR experiments	High for the complete recipe; masked pretraining, Global Response Normalization and downstream training enlarge the scope.
DRAEM, <i>A Discriminatively Trained Reconstruction Embedding for Surface Anomaly Detection</i> [6]	ICCV 2021; deep reconstruction/autoencoder and discriminative networks	Industrial defect detection/localization; MVTec AD [7]	Moderate to high; synthetic anomaly generation, reconstruction and pixel-level evaluation on larger images. Older, but

Candidate and primary paper	Year and eligibility	Application and datasets	Difficulty and local feasibility
			within the 2020s criterion.
ConvNeXt, A <i>ConvNet for the 2020s</i> [2]	CVPR 2022; deep CNN, recorded in the implementation-stage comparison	Classification; CIFAR-10/CIFAR-100	Moderate; modern convolutional architecture requiring downscaling. Distinct from ConvNeXt V2.

The initial response also mentioned a generic compact convolutional autoencoder. Without a specific recent paper, however, it did not fully satisfy the selection criteria. The table therefore includes only identifiable published methods. Its cost comparisons are qualitative assessments rather than local benchmarks.

Selection rationale

StarNet was recommended because it has a recent peer-reviewed paper, a clearly defined CNN architecture, an official implementation and a focused question for ablation. Its 2024 publication satisfies the assignment's 2020s criterion, although this does not make it the newest model in 2026 or the best model for every task. Its compact architecture and use of standard layers suit the M5/16 GB computing budget.

CIFAR-10 [4] offers 60,000 public 32x32 RGB images, ten classes, a standard 50,000/10,000 train/test split and an approximately 163 MB Python archive. It permits training from scratch and reproducible classification evaluation. CIFAR-100 would increase task difficulty, while MVTec AD suits DRAEM but adds localization and data-handling complexity. Full ImageNet training was outside the intended local scope. I approved the proposed 45,000/5,000/10,000 split, comparable CNN baseline, star-operation ablation, learning curves, confusion matrix and timing analysis. Primary sources are listed in section 6; the conversation establishes the initial selection history, while docs/research.md records the implementation-stage research.

2 Algorithm description

StarNet was introduced by Xu Ma, Xiyang Dai, Yue Bai, Yizhou Wang and Yun Fu in *Rewrite the Stars*, published at CVPR 2024 [1]. The paper investigates whether element-wise multiplication of learned feature projections can improve the representational capacity of compact networks. StarNet is a deep CNN: learned convolutions perform spatial mixing, pointwise convolutions perform channel mixing, and successive convolutional blocks form a hierarchy. It is neither a Vision Transformer nor an autoencoder.

Star operation and data flow

For block input x , let $z = \text{BN}(\text{DWConv}_{7 \times 7}(x))$. Two pointwise convolutions produce $a = W_1 z + b_1$ and $b = W_2 z + b_2$, each with three times the input channels in this adaptation. The block computes $h = \text{ReLU}_6(a * b)$, where $*$ is element-wise multiplication. It projects h back to the original width, applies batch

normalization and a second 7x7 depthwise convolution, then adds the residual input x . Thus the block preserves spatial dimensions and channel count.

Ignoring bias and activation for illustration, $(w_1^T z)(w_2^T z) = \sum_i \sum_j w_{1_i} w_{2_j} z_i z_j$. This expression reveals pairwise feature interactions without explicitly storing every quadratic feature. The paper relates these interactions to implicit feature spaces in kernel methods. Because the implemented network also includes ReLU6, normalization and residual connections, the equation provides intuition rather than defining a literal kernel algorithm or guaranteeing better accuracy. The two projections constrain the learned coefficients, so the coefficient of each product term cannot vary independently.

Component	Original main StarNet family	CIFAR-10 adaptation
Input and output	ImageNet images and 1000 classes	32 x 32 RGB and 10 classes
Stem	3x3 convolution, stride 2, 32 channels	3x3 convolution, stride 1, 24 channels
Stages	Four stages, widths double	Three stages: 24 / 48 / 96
Depth and stride	Variant-dependent depth; stride 2 per stage	Depth 1 / 1 / 2; stride 1 / 2 / 2
Branch expansion	4 in main paper variants	3, also used by some official small variants
Head	Batch norm, global average, classifier	Same structure with 10 outputs
Training	Original ImageNet recipe	Local budget; no pretrained weights

The adaptation retains stage outputs of 32x32, 16x16 and 8x8 to avoid excessive downsampling of small images. It uses no stochastic depth and applies PyTorch's default initialization for convolutional and linear layers. It therefore does not reproduce the original initialization or training recipe exactly. The objective is mean ten-class cross-entropy, $L = -(1/N) \sum_i \log \text{softmax}(f(x_i))[y_i]$.

The baseline has two 3x3 Conv-BN-ReLU layers per stage, widths 32/64/128, 2x2 max pooling after each stage, global average pooling and a linear head. The ablation retains every StarNet layer and changes only $\text{ReLU6}(a) * b$ to $\text{ReLU6}(a) + b$. The two StarNet variants therefore have exactly the same number of parameters.

3 How AI implemented the algorithm

Codex assisted with research, design, coding, command execution, debugging, experiments, visualization and report preparation. My role was to specify requirements, assess the recommendation, approve StarNet/CIFAR-10 and request completion of the assignment. The implementation-stage notes also record the supplied GitHub destination. I did not manually write the project code. The recorded conversation documents the research and approval stages, while the saved project artifacts and verification records support the implementation account below.

The first implementation attempt stopped because usable file-editing and command-execution tools were unavailable, and it produced no project files or experiments. The completed runs described below are documented by artifacts from the later implementation stage. This report revision draws on saved results and source files; training was not rerun.

From paper to executable pipeline

According to the implementation-stage records, both `requirement.txt` and `requirement_2.txt` were available when implementation began, whereas only the first file had been present during the initial research inspection. Checks of Python packages, storage and hardware showed that system Python lacked the ML stack, Anaconda provided PyTorch and torchvision, and MPS became available outside the sandbox. The saved execution environment identifies MPS as the training device. The approved three-model protocol was implemented before full training.

`src/models.py` implements the official StarNet block using standard PyTorch layers, with a single selector for multiplication or addition. `src/data.py` downloads CIFAR-10, creates the seeded stratified split, computes normalization statistics from the training data only, and prepares augmented training and deterministic evaluation datasets. `src/train.py` constructs the optimizer and scheduler, runs training epochs, writes histories atomically and saves the best validation checkpoint. It then reloads that checkpoint for evaluation on the official test set.

`scripts/analyze.py` converts saved predictions and histories into figures, CSV summaries and dashboard JSON. The dashboard in `website/` loads this JSON and the generated images, with controls for selecting metrics, models and prediction outcomes. `scripts/verify.py` reconstructs confusion matrices and accuracy, checks that split indices are disjoint, verifies parameter structures, reloads checkpoints on CPU and checks that forward outputs are finite.

The original report was generated from these results by `scripts/report.py`. This Markdown version also incorporates the earlier research conversation. The existing `report_content.json` and Word/PDF files have not been regenerated as part of this editorial revision. Running `scripts/report.py` would regenerate the report from its existing inputs rather than preserve these editorial changes.

Website and evidence traceability

The website is a local interactive dashboard served from the project root. It displays saved experiments from `artifacts/results/dashboard.json` and generated images, alongside descriptions of the architecture and dataset. Users can select models and metrics to view comparisons and learning curves, or filter predictions to examine correct and incorrect examples. The dashboard does not launch new training. Timing values follow the measurement definitions in section 4.

Report or website content	Saved evidence
Accuracy, loss, parameter count and timing	<code>artifacts/results/*_results.json</code> and <code>experiment_summary.csv</code>
Training and validation curves	<code>artifacts/results/*_history.json</code>
Confusion matrices and prediction examples	<code>artifacts/results/*_predictions.json</code> and generated figures
Protocol and environment	<code>configs/full.json</code> , <code>artifacts/results/config.json</code> , <code>split.json</code> and <code>environment.json</code>
Metric/checkpoint and controlled-ablation checks	<code>artifacts/results/verification.json</code> and <code>protocol_checks.json</code>

Report or website content	Saved evidence
Recorded website checks	artifacts/results/website_http_checks.json and website_ui_checks.json

The saved verification records document successful metric reconstruction and checks of checkpoints, data splits and the website. These checks were completed previously and were not rerun during this Markdown revision.

Pilot verification and corrections

The required pilot used 2048 training, 512 validation and 512 test examples for two epochs per model. It tested the pipeline from downloading and data loading through forward and backward passes, loss computation, optimization and validation. It also checked saving, loading, evaluation, logging and figure generation. After corrections, training losses decreased and validation accuracy exceeded chance level. Combined model training took about nine seconds, excluding preparation and the roughly five-minute dataset download. Pilot results are stored separately in [artifacts/pilot](#) and are not used as final results.

The pilot test subset comprised the first 512 official test images, resulting in limited exposure to the test set during pipeline verification. The complete test set therefore cannot be described as entirely unseen. Pilot test scores were not used to tune accuracy, select checkpoints or determine the full training budget.

Several corrections were made before full training. An assumed path to the official source returned 404; inspecting the GitHub repository tree located [imagenet/starnet.py](#). Normalization reductions were changed to float64 accumulation to improve numerical accuracy. The initial baseline was widened to match StarNet's parameter budget, and its pilot was rerun. A writable plotting cache resolved font-cache warnings. The final pilot passed independent verification and produced eleven figures before full training began.

The implementation omits additional training techniques that would complicate the operator comparison: pretrained weights, mixup, distillation and model-specific tuning. Fixed seeds improve repeatability, although MPS execution is not guaranteed to be bitwise deterministic. Checkpoint and training-source hashes document the provenance of the saved runs.

4 Experiment settings and results

Experiment settings

CIFAR-10 [4] is a public image classification dataset with 60000 color images, ten mutually exclusive classes and 32x32 resolution. The official training set contains 50000 images; 500 per class were held out for validation, leaving 4500 per class for training. The official 10000-image test set has 1000 images per class. No validation or test examples contribute to gradient updates or normalization statistics.

Setting	Actual value
Split and seed	45000 train / 5000 validation / 10000 test; seed 3024
Input and augmentation	32x32 RGB; 4-pixel padding, random crop, horizontal flip for training only

Setting	Actual value
Normalization mean	0.491045, 0.481686, 0.445724
Normalization standard deviation	0.247093, 0.243491, 0.261444
Hardware	Apple M5; 16 GiB unified memory; MPS
Operating system	macOS-26.6.2-arm64-arm-64bit-Mach-O
Software	Python 3.14.6; PyTorch 2.14.0; torchvision 0.29.0; NumPy 2.4.6
Optimizer	AdamW, initial lr 0.003, weight decay 0.0005; betas 0.9/0.999, eps 1e-8
Schedule and budget	CosineAnnealingLR to zero after 20 epochs; no warmup; batch 128; no early stopping
Loss and precision	Mean cross-entropy; float32; no label smoothing
Checkpoint selection	Highest validation accuracy; earliest epoch on ties
Evaluation	Reload selected checkpoint; eval mode; no gradients or test augmentation
Randomness	Python, NumPy, PyTorch, MPS and loader seeds fixed; zero loader workers



Figure 1. One actual training example per class from the saved training split.

Classification results and computational cost

Model	Test accuracy	Test loss	Best val.	Selected epoch
Conventional CNN	87.22%	0.3875	87.58%	20
StarNet	86.14%	0.5098	86.46%	19

Model	Test accuracy	Test loss	Best val.	Selected epoch
Additive ablation	85.60%	0.4340	85.68%	20

Model	Parameters	Conv/linear MACs	Train min	ms / image
Conventional CNN	289,194	38.63 M	4.22	0.0570
StarNet	280,522	37.33 M	10.95	0.1223
Additive ablation	280,522	37.33 M	10.43	0.1237

Table values come from [artifacts/results/*_results.json](#) and [experiment_summary.csv](#). Training time includes validation and checkpoint writing but excludes downloading, data preparation and final testing. Inference was measured using batches of zero-valued inputs already resident on the device, with ten warmups and thirty synchronized measurements. The reported time per image is the median batch time divided by batch size, an amortized throughput cost rather than single-image latency. Data transfer and preprocessing are excluded. These measurements use unfused eager PyTorch execution on MPS and are not directly comparable to the paper's ONNX/CoreML deployment benchmarks.

MACs count convolutional and linear multiply-accumulate operations for one 32x32 image. They exclude biases, normalization, activation, pooling and element-wise branch operators, so they represent a partial operation count rather than total FLOPs. The equal MAC counts for StarNet and the additive ablation follow from this counting convention and do not imply identical executed instructions.

Saved checkpoint sizes: Conventional CNN 1.121 MiB; StarNet 1.111 MiB; Additive ablation 1.112 MiB. These serialized files contain model state and metadata, not optimizer state. The baseline has 3.09% more parameters than StarNet; the capacity comparison is close, but receptive fields and optimization differ.

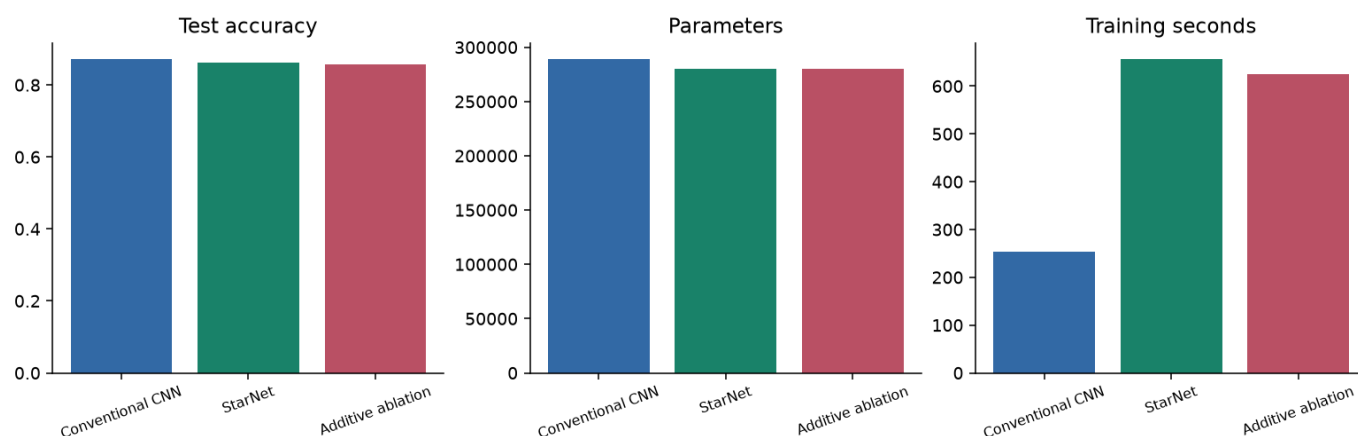


Figure 2. Actual test accuracy, capacity and measured training duration.

StarNet's test accuracy difference relative to the baseline is -1.08 percentage points. This comparison evaluates the two architectures under a shared training budget but does not isolate the effect of the star operator. The additive ablation provides a more controlled comparison of that operation.

Learning curves

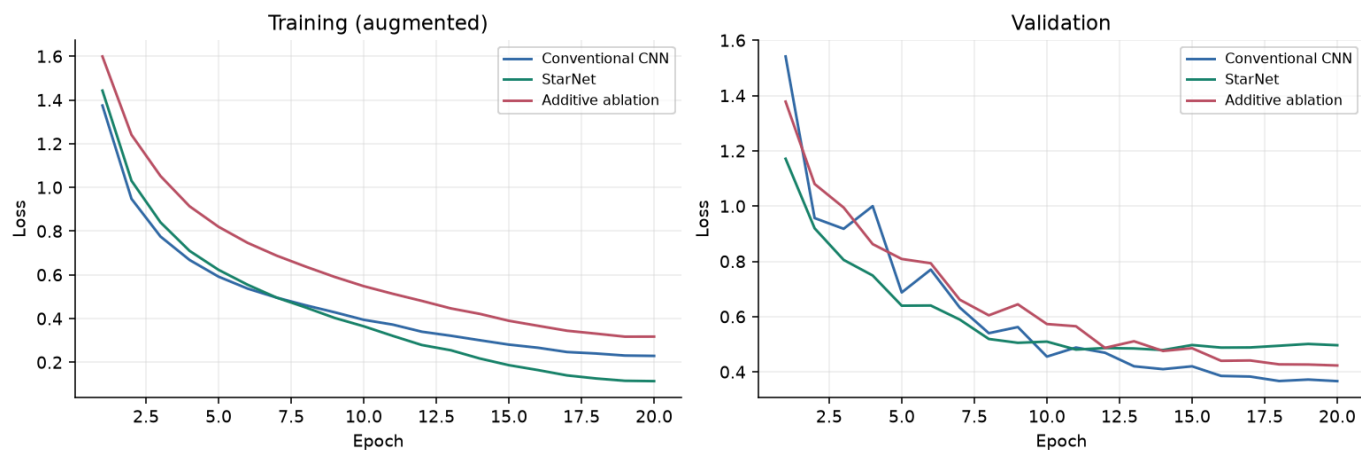


Figure 3. Training and validation cross-entropy, generated from each saved epoch history.

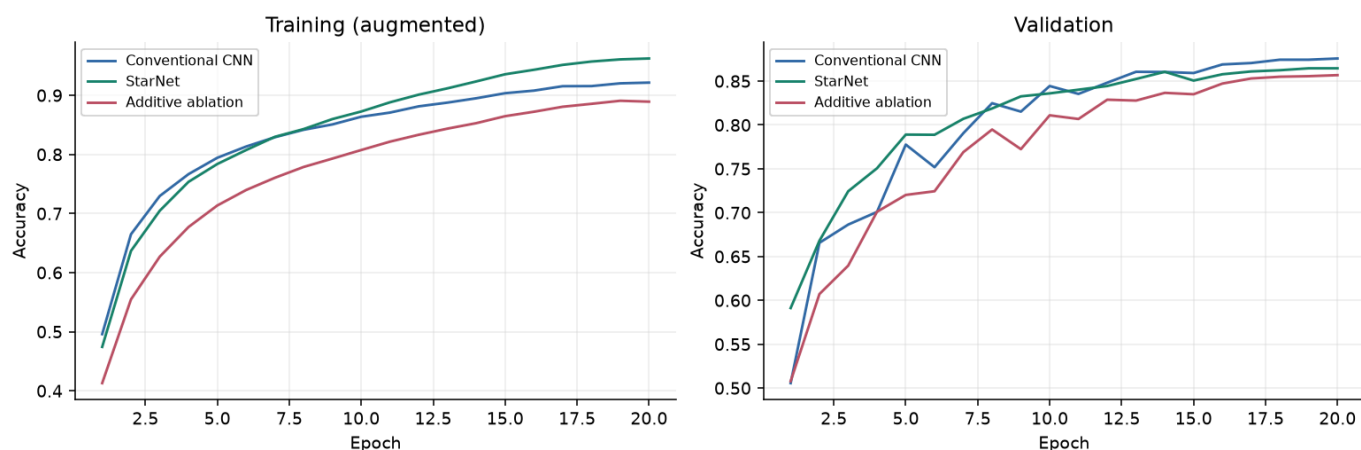


Figure 4. Training and validation accuracy for the same runs.

Training metrics are computed across augmented batches while model parameters change within each epoch. Validation metrics are measured afterward in evaluation mode without augmentation. The curves therefore reflect different input distributions and evaluation conditions. Batch-normalization statistics and the initial learning rate can contribute to validation fluctuations. Selecting checkpoints by validation accuracy avoids automatically choosing the final epoch.

At the final epoch, the conventional CNN achieved training accuracy of 92.15% and validation accuracy of 87.58%; StarNet achieved 96.23% and 86.46%, respectively; and the additive ablation achieved 88.92% and 85.68%, respectively. The training budget was fixed before the full experiments and was not extended in response to test results. These runs do not establish the best attainable accuracy of each model.

StarNet's final training-minus-validation accuracy gap is 9.77 percentage points, compared with 4.57 for the baseline. The larger gap and lower held-out accuracy are consistent with a closer fit to the training set without improved generalization. However, the different training and evaluation conditions prevent this gap from serving as a pure measure of overfitting. These observations do not establish a causal explanation.

Confusion matrix and class errors

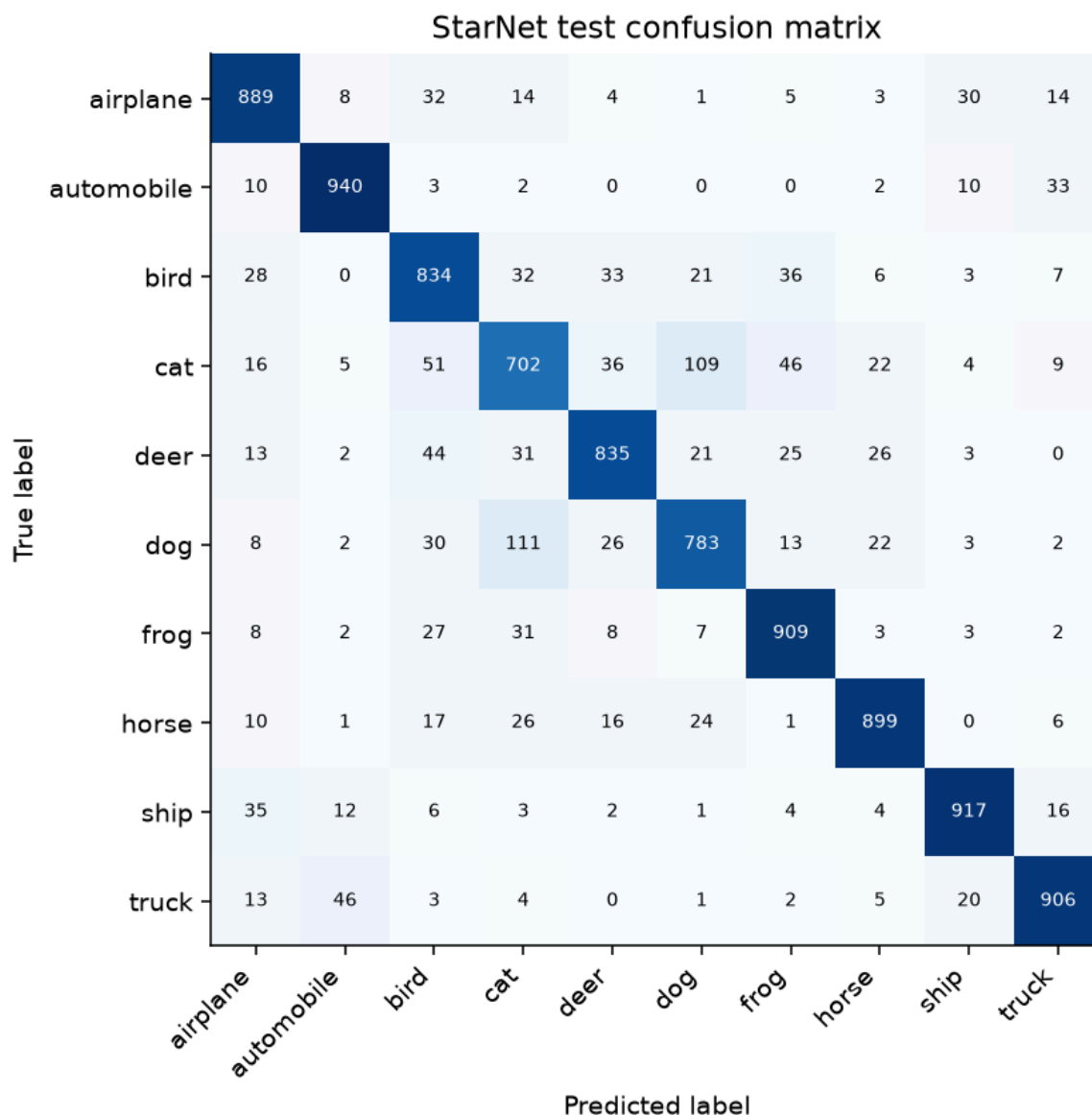


Figure 5. StarNet official-test confusion matrix. Rows are true labels; columns are predictions.

The largest off-diagonal counts are dogs predicted as cats (111), cats predicted as dogs (109), and cats predicted as birds (51). These counts describe observed label confusions but do not identify the visual mechanisms behind the errors. Low image resolution and similarities between classes are plausible contributing factors, although further analysis would be needed to establish their effects.

StarNet's recall ranges from 70.20% for cat to 94.00% for automobile. Because each class has 1000 test images, overall accuracy equals macro-averaged recall in this evaluation. Full confusion matrices and per-class recall for all models are available in the result files and on the website.

Ablation and representative predictions

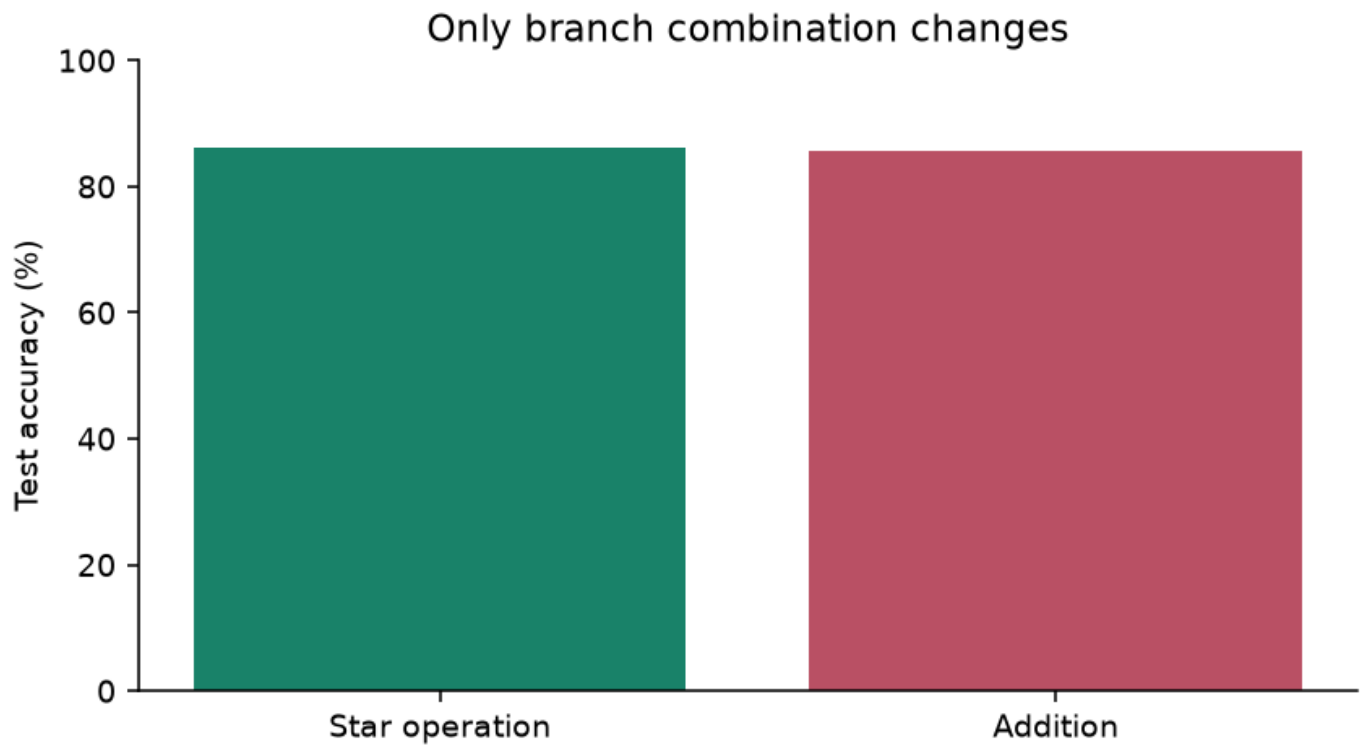


Figure 6. Multiplication versus addition with identical StarNet parameter structure.

StarNet achieved 86.14% test accuracy, compared with 85.60% for the additive ablation, a signed difference of +0.54 percentage points. This run supports a benefit from the star operation under the selected protocol. Widths, depths, branches, initialization seed, data split, augmentation, optimizer and epoch budget were held fixed. Replacing multiplication with addition also changes activation scale, so the observed difference includes the associated optimization effects. A single seed cannot establish statistical significance or universal superiority.



Figure 7. First six correct and first six incorrect StarNet predictions in official test order. T is true label; P is predicted label.

The examples follow a deterministic selection rule and do not provide a random estimate of error prevalence. All test predictions, true labels and maximum softmax probabilities are saved. The website allows users to switch models and filter correct or incorrect predictions. Because softmax confidence has not been calibrated, it should not be interpreted as a guaranteed probability of correctness.

5 What I learned from this AI assignment

This reflection was prepared with AI assistance and is grounded in the documented project evidence. It focuses on the assignment's technical lessons. As described in section 3, Codex wrote the code and executed the commands.

I learned to define algorithm-search requests using criteria that can be checked. A recent publication date alone does not establish that a method is a deep CNN, and a familiar name does not demonstrate local feasibility. Reading the primary paper and official implementation clarified StarNet's convolutional structure and the role of element-wise multiplication. The selection-stage comparison of FasterNet, ConvNeXt V2 and DRAEM, together with the later consideration of ConvNeXt, highlighted the importance of manageable complexity and a clear ablation question for a student project. Keeping the original research request and explicit approval also provided a more reliable record than reconstructing the conversation afterward.

The project also taught me to distinguish an AI's intended actions from completed work. Although progress messages described planned steps, the initial attempt could not produce an implementation because the required tools were unavailable. Saved experiment artifacts were therefore essential to support the results in this report. Similarly, incorporating the earlier conversation corrected the previous draft's claim that no search conversation was available. Evaluating AI-generated documentation requires checking both its claims and their sources.

Studying the architecture clarified why the star operation involves more than applying a nonlinear activation. Multiplying two learned projections introduces products between feature coordinates. Depthwise convolutions mix spatial information within channels, while pointwise convolutions mix information across channels. The residual path retains the input and supports stacking blocks. Understanding these distinct roles made it easier to read the block diagram and check whether the generated code implemented the intended computation.

Adapting the model to CIFAR-10 showed why architectural changes must be described precisely. Reusing an ImageNet downsampling pattern on 32x32 images would quickly reduce spatial resolution. The stride-1 stem, fewer stages and smaller classifier are practical adaptations, but they also mean that this project is not an exact reproduction of the paper. Similarly, matching parameter counts improves the baseline comparison without making the networks identical in computation, receptive field or optimization difficulty.

The controlled operator comparison yielded a +0.54-percentage-point difference in test accuracy for multiplication relative to addition. This result taught me to distinguish a specific observation from a general claim about all networks. A theoretical argument for richer features does not determine accuracy across datasets or training budgets. Changing one operation can also affect feature scale and optimization, so an ablation must state both what is held fixed and what else changes with the operation.

The pilot demonstrated why AI-generated code needs to be checked through execution. The first environment check did not expose the GPU, an assumed upstream path was incorrect, and the initial baseline was too small for the intended comparison. Hardware checks, repository inspection, numerical review and a repeated pilot resolved these issues. Reloading checkpoints, checking that outputs were finite and reconstructing metrics provided stronger evidence than saving checkpoints alone.

Finally, I learned to distinguish the roles of validation and testing. Validation accuracy determined checkpoint selection; the official test set was evaluated after training and did not determine the training budget. Reproducibility also requires split indices, preprocessing statistics, seeds, software versions and

clear timing definitions. The study's main limitations are its single seed, short training budget, single dataset and uncalibrated confidence. Multiple seeds and longer fixed-budget comparisons would be useful future experiments, while the present conclusions remain limited to the reported protocol.

6 Source code repository

The project source code is available at:

<https://github.com/yuannnnn1/StarNet>

The repository provides access to the project code. The local project contains the complete implementation and reproduction instructions. The official StarNet repository cited below is a separate upstream source.

Repository structure and reproduction

The project is organized by function. `src/` contains the model, data, training and evaluation modules; `configs/` contains the pilot and full experiment protocols; `scripts/` contains analysis, verification and report-generation tools; `website/` contains the dashboard; and `docs/` records research and checks. Outputs are stored in `artifacts/results/` for machine-readable metrics and per-image predictions, `artifacts/figures/` for plots, `artifacts/checkpoints/` for local model states, and `artifacts/report/` for the report. Large datasets, environments and checkpoints are excluded from Git.

Install the Python dependencies listed in `requirements.txt` in a virtual environment. The measured environment is documented in section 4 and `artifacts/results/environment.json`.

Run `python -m src.train --config configs/pilot.json` for the pilot, followed by `python -m scripts.analyze --root artifacts/pilot` and `python -m scripts.verify --root artifacts/pilot`. For the full experiments, run `python -m src.train --config configs/full.json`. To evaluate saved checkpoints, run `python -m src.train --config configs/full.json --evaluate-only`. Use `python -m scripts.analyze` and `python -m scripts.verify` to generate figures and check consistency.

To view the dashboard, serve the project root with `python -m http.server 8000 --bind 127.0.0.1` and open `http://127.0.0.1:8000/website/`. The command `python -m scripts.report` generates the Word report. `README.md` provides the complete setup and rendering procedure. CIFAR-10 is downloaded from its official source through torchvision and is not redistributed in the repository. Checkpoints are retained locally for evaluation and can be regenerated by training.

References

[1] Xu Ma, Xiyang Dai, Yue Bai, Yizhou Wang and Yun Fu. Rewrite the Stars. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 5694–5703.

[CVPR 2024 primary paper](#)

[Official implementation at commit c999eb5](#)

[2] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell and Saining Xie. A ConvNet for the 2020s. CVPR, 2022, pp. 11976–11986.

[ConvNeXt primary paper](#)

[3] Jierun Chen, Shiu-hong Kao, Hao He, Weipeng Zhuo, Song Wen, Chul-Ho Lee and S.-H. Gary Chan. Run, Don't Walk: Chasing Higher FLOPS for Faster Neural Networks. CVPR, 2023, pp. 12021-12031.

[FasterNet primary paper](#)

[4] Alex Krizhevsky. Learning Multiple Layers of Features from Tiny Images. Technical report, University of Toronto, 2009. Dataset creators: Alex Krizhevsky, Vinod Nair and Geoffrey Hinton.

[Official CIFAR-10 dataset and report](#)

[5] Sanghyun Woo, Shoubhik Debnath, Ronghang Hu, Xinlei Chen, Zhuang Liu, In So Kweon and Saining Xie. ConvNeXt V2: Co-Designing and Scaling ConvNets With Masked Autoencoders. CVPR, 2023, pp. 16133-16142.

[ConvNeXt V2 primary paper](#)

[6] Vitjan Zavrtanik, Matej Kristan and Danijel Skocaj. DRAEM: A Discriminatively Trained Reconstruction Embedding for Surface Anomaly Detection. ICCV, 2021, pp. 8330-8339.

[DRAEM primary paper](#)

[7] MVTec Software. MVTec AD industrial anomaly detection dataset. The official page describes more than 5,000 images across fifteen categories, normal training images and pixel-level test annotations.

[Official MVTec AD dataset](#)